CO1 Assessment Task 1

Cloud Computing Fundamentals — Concept Mapping & Applied Architecture

CO1 • BL2 (Apply)	**Weightage** • 20%	**Dave's Taxonomy** • Imitation (L1)	**Total Marks** • 50

APPLIED PROJECT

Campus Connect — Smart Student Engagement & Learning Platform

A web + mobile application for attendance tracking, campus event management, and e-learning resource delivery, backed by a microservice cloud architecture with an AI-based recommendation module.

Note: Replace "Campus Connect" with your own registered project name/modules throughout this document before final submission, per the assignment's customization requirement.

Code of Conduct

I **P. Amrutha (Reg. No. 192372359)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P. Amrutha Date: 17-07-2026

TASK 1 Concept Map — Cloud Computing Ecosystem

The concept map below connects the cloud ecosystem, cloud actors, service models, deployment models, virtualization, web services, REST/SOAP APIs, elasticity, scalability, and the Campus Connect project into a single hierarchical diagram, radiating outward from the central concept.

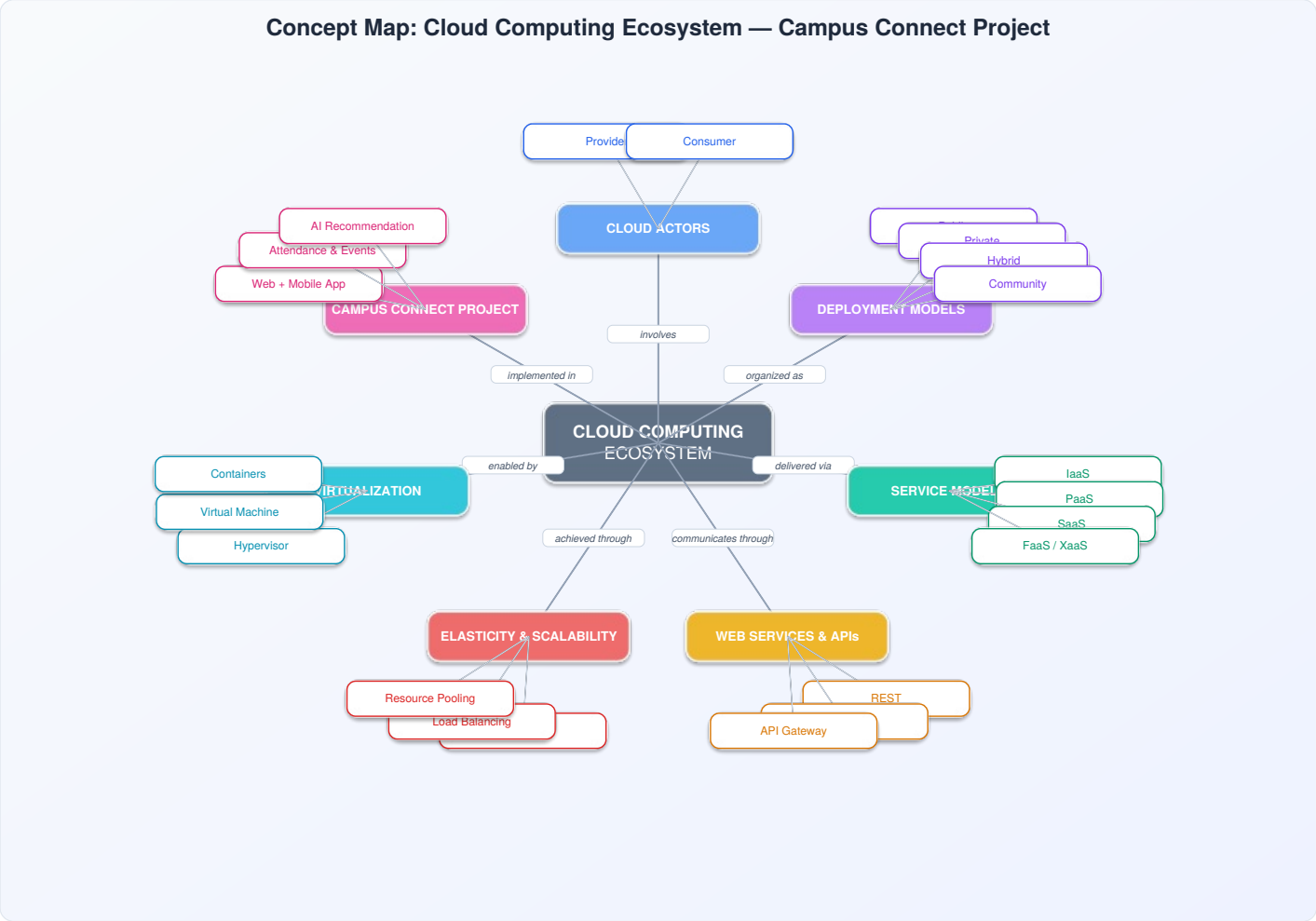

Figure 1. Radial concept map — center node (Cloud Computing Ecosystem), seven category nodes, and their associated leaf concepts, connected with labeled relationship links.

Reading the map: Each spoke from the center carries a relationship label (e.g. "*delivered via,*" "*enabled by,*" "*communicates through*") describing how that category relates to the ecosystem as a whole. Leaf nodes attached to each category represent the concrete concepts a cloud architect chooses between when designing a real system such as Campus Connect — for example, the project settles on a hybrid mix of PaaS and FaaS under Service Models, and REST over API Gateway under Web Services & APIs.

Key Nodes Summary

- **Cloud Actors:** Provider (infrastructure owner) and Consumer (Campus Connect's engineering team and end users).
- **Deployment Models:** Public, Private, Hybrid, Community — Campus Connect uses a public cloud with a private VPC subnet for sensitive student data.
- **Service Models:** IaaS, PaaS, SaaS, FaaS/XaaS — mapped in detail in Task 2A.
- **Web Services & APIs:** REST (primary), SOAP (legacy university ERP integration), API Gateway (single entry point).
- **Elasticity & Scalability:** Auto-scaling groups, load balancing, resource pooling — visualized in Task 3.
- **Virtualization:** Hypervisor, virtual machines, and containers as the enabling layer beneath every service model.

TASK 2A Service Model Responsibility Matrix

The stack diagram below shows, layer by layer, what shifts from the Campus Connect team's responsibility to the cloud provider's as we move from on-premises infrastructure toward IaaS, PaaS, FaaS, and SaaS.

Figure 2. Shared responsibility stack across service models. Colored blocks = provider-managed layers; white blocks = layers Campus Connect's team still manages.

Service Category	Managed by Cloud Provider	Managed by Campus Connect Team	Why This Split Fits the Project
SaaS e.g. hosted email/office suite	Application code, runtime, OS, middleware, availability, patching	User accounts, data entered into the app, access permissions	Generic office/communication tools don't need custom engineering — buying them frees the team to focus on student-facing features.
PaaS backend microservices	OS, runtime, patching, scaling infrastructure, load balancer	Application code (attendance/event logic), business rules, deploy configs	Backend features change every sprint; PaaS lets the team ship without managing servers.
IaaS AI recommendation module	Physical hardware, virtualization, network, base storage	OS patching, ML runtime/libraries, model code, security groups	The recommendation engine needs custom CPU sizing and Python/ML libraries that PaaS would restrict.
FaaS / Serverless push notifications, image resize	Everything except function code and trigger configuration	Function logic, event triggers, memory/timeout settings	Bursty, event-driven tasks (e.g. exam-week alert spikes) scale instantly without paying for idle compute.
Database-as-a-Service	DB engine install, patching, replication, automated backups, failover	Schema design, query optimization, indexing, access control	Removes DBA overhead while guaranteeing high availability for attendance and student records.
Storage-as-a-Service	Durability, redundancy, storage hardware, geo-replication	Bucket policies, folder structure, lifecycle rules, CDN linkage	Course media and uploads are large and rarely modified — cheaper and more durable than DB blobs.
Identity-as-a-Service	Identity store, MFA, token issuance, compliance certifications	Role mapping, login UI, in-app session handling	Authentication is security-critical and reusable; buying it avoids reinventing password storage.
Monitoring / Logging-as-a-Service	Log ingestion pipeline, storage, dashboard hosting	What to log, alert thresholds, dashboard views	Gives near real-time visibility into attendance-sync failures or API errors without building a logging platform.

TASK 2B Cloud Service Decision Matrix

Each Campus Connect module is mapped to the cloud service category best suited to it, with the reasoning behind the choice rather than forcing every possible service into the design.

Project Module	Recommended Cloud Service	Justification
Authentication	Identity-as-a-Service (managed auth/SSO provider)	Single sign-on across web and mobile, built-in MFA, reduces breach risk for student PII.
Frontend Hosting	Static hosting + CDN	React build served from edge locations for low latency across the campus and off-campus users.
Backend / API Layer	Container-based PaaS running independent microservices	Attendance, Event, and Notification services scale independently based on their own load patterns.
Database	Managed relational Database-as-a-Service	ACID compliance needed for attendance and grade-linked records; automated failover.
Object Storage	Cloud object storage bucket	Durable, low-cost storage for course material uploads, event banners, profile photos.
Analytics / Logging	Centralized logging & analytics service	Tracks usage patterns, API latency, and error rates across all microservices in one place.
Notification	Serverless functions + push/email delivery service	Event-driven and bursty (class reminders, exam alerts) — ideal for pay-per-invocation FaaS.
AI / Python Module	IaaS virtual machine (GPU-capable tier)	Custom ML environment for the event-recommendation and attendance-anomaly-detection model.
Security	WAF + IAM roles + encryption at rest/in transit	Protects student PII and mitigates common web attacks (SQLi, XSS, credential stuffing).
CI/CD	Managed CI/CD pipeline (e.g. GitHub Actions + cloud deploy step)	Automated build/test/deploy on every commit reduces manual release errors.
Backup	Automated snapshot service	Point-in-time recovery for the database and storage layer in case of failure or data loss.
Monitoring	Cloud monitoring/dashboard service with alerting	Real-time system health visibility and on-call alerting for the engineering team.

TASK 3 API Flow, Elasticity & Scalability Design

The diagram shows the full request path from client to data layer, the security boundary, the auto-scaling compute group, and the supporting analytics/monitoring and CI/CD flows.

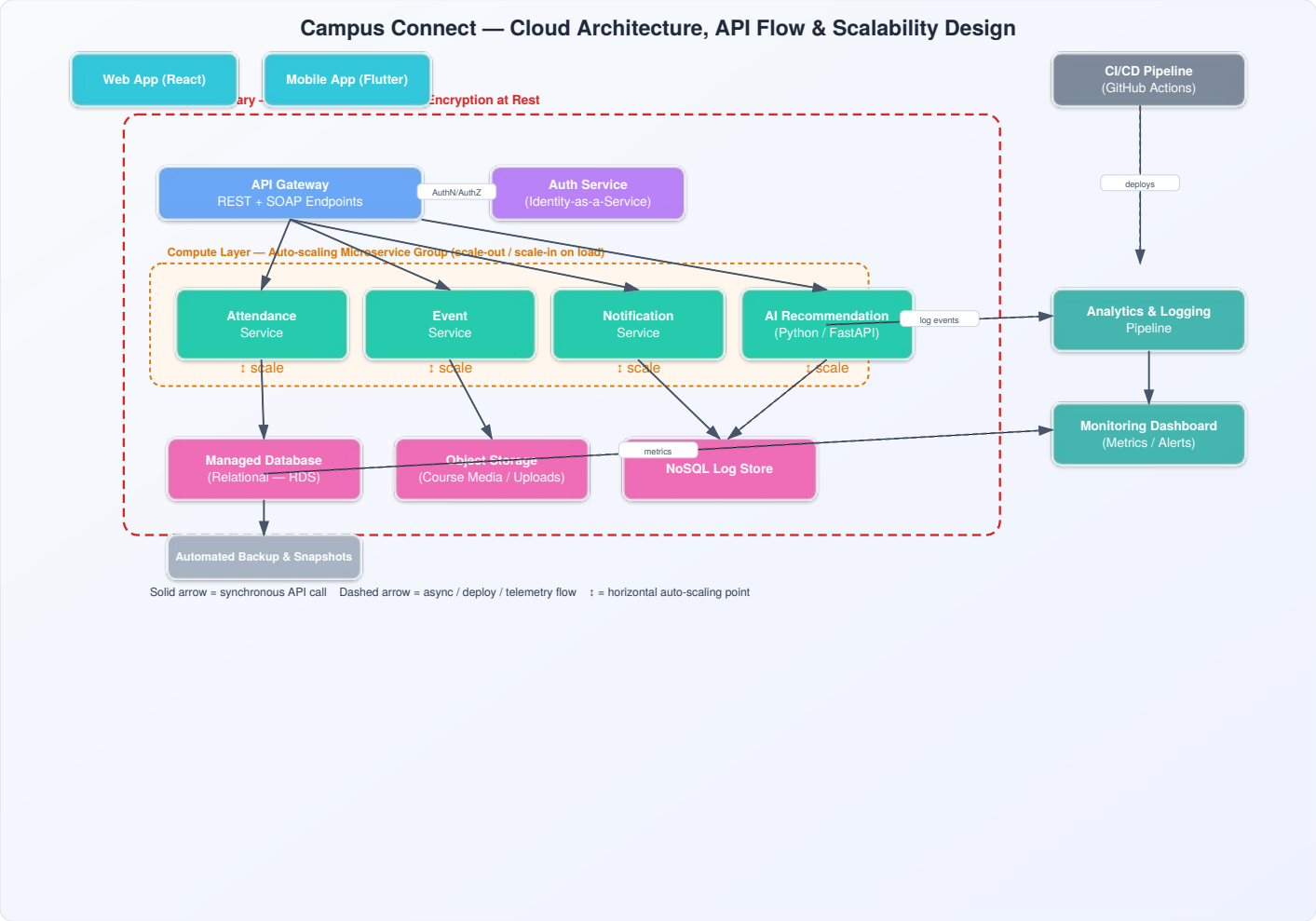

Figure 3. Campus Connect cloud architecture — client apps, API Gateway, Auth service, an auto-scaling microservice compute layer, data layer, and monitoring/CI-CD support flows, all inside a defined security boundary.

Scaling points: the four microservices inside the auto-scaling group (Attendance, Event, Notification, AI Recommendation) scale out horizontally and independently — e.g. the Attendance service scales up sharply during first-period class hours, while the AI Recommendation service scales up around event-registration windows, without over-provisioning the other services.

TASK 4 Technical Justification

Together, the concept map, the two responsibility/decision matrices, and the architecture diagram demonstrate CO1 by tracing a cloud solution from first principles to a working design. The concept map anchors the vocabulary (service models, deployment models, virtualization, APIs, elasticity) that the matrices and diagram then apply concretely to Campus Connect. The service model choice is not uniform: PaaS is used where iteration speed matters, IaaS where the AI module needs low-level control, and FaaS where workloads are event-driven and bursty — showing a deliberate, workload-aware selection rather than a default choice. APIs (REST at the gateway, with a legacy SOAP bridge for the university ERP) give the client apps a stable, versioned contract independent of how the backend is implemented, which is what allows the compute layer to scale or be re-architected without breaking the frontend. Elasticity and scalability are realized through the auto-scaling microservice group and the load balancer in front of it, so each service absorbs its own traffic pattern instead of the whole system scaling as one block. Virtualization/managed compute choices (containers on a PaaS layer, a dedicated VM for the ML workload) balance operational simplicity against the control the AI module needs. Finally, the security boundary drawn around the gateway, compute, and data layers — combined with the shared-responsibility split in Task 2A — makes explicit which risks the cloud provider absorbs and which remain Campus Connect's to manage.

1. Evaluate the effectiveness of your concept map in representing cloud computing principles. Which design decisions most strongly support your solution, and why?

The radial hub-and-spoke layout works well because it keeps the seven core categories equally visible instead of implying a false linear order between them. The strongest design decision was labeling every link (e.g. "delivered via," "enabled by") rather than leaving plain lines — this forces precision about *how* concepts relate, not just that they do, which is exactly what CO1 asks students to demonstrate at the "Apply" level. Tying one branch directly to the Campus Connect project also grounds otherwise abstract terms (elasticity, virtualization) in a concrete system, making the map useful as a design reference rather than a static glossary.

2. Analyze how scalability, elasticity, and resource management influence the architecture shown in your concept map. What challenges could arise if these capabilities were absent?

Scalability and elasticity are what let the four microservices in Task 3 grow or shrink independently based on their own demand curves, and resource pooling is what makes that economical — idle capacity from one service can effectively be reused by another at the infrastructure level. Without these capabilities, Campus Connect would need to provision every service for its peak load at all times, which is costly and still fragile: a single traffic spike (e.g. attendance check-in at 9 a.m.) could exhaust fixed capacity and degrade or crash unrelated services like notifications, since there would be no isolation or elastic headroom to absorb the spike.

3. Justify the selection of the APIs, web services, or cloud services included in your concept map. How do they contribute to the overall functionality and efficiency of the system?

REST was chosen as the primary API style for its statelessness and lightweight JSON payloads, which suit the frequent, small requests mobile clients make (marking attendance, fetching event lists). SOAP is retained only at the boundary with the legacy university ERP system, where it is already the required contract — introducing it everywhere would add unnecessary overhead. The API Gateway centralizes routing, authentication checks, and rate limiting so individual microservices don't duplicate that logic, improving both efficiency and consistency across the system.

4. Critically examine the relationships between the components in your concept map. How do these interactions improve reliability, performance, or user experience?

The relationship between virtualization and the service models is foundational: containers and VMs are what let PaaS and IaaS exist at all, and that separation is what lets Campus Connect swap or scale individual services without touching the others, improving reliability through isolation. The link between the API Gateway and the auto-scaling compute layer improves performance directly, since the gateway can load-balance requests across however many instances are currently running. The link from monitoring/logging back into the compute layer closes the loop operationally — it is what lets the team detect and react to a scaling or failure event before it visibly affects the user experience.

5. Reflect on the process of developing your concept map. How has creating and analyzing the map changed your understanding of cloud computing architecture, and what improvements would you make if you were redesigning it?

Building the map made clear that service model selection is a set of trade-offs made per workload, not a single decision for the whole system — something that was easy to state abstractly before but only became concrete once every Campus Connect module had to be assigned a specific model and justified. If redesigning it, a next version would add a small "cost/control trade-off" axis alongside each service model branch, since that trade-off is the real reason IaaS, PaaS, and FaaS were each chosen for different modules, and making it visible on the map itself would make the reasoning even more explicit at a glance.

README

Repository Documentation

The following README accompanies the GitHub repository submission alongside the editable diagram sources and exported evidence.

```
# CSA15_CO1_AT1_<RegisterNumber>

## Title
Cloud Computing Fundamentals — Concept Map, Responsibility & Decision Matrices,
and Architecture Design for the Campus Connect Project

## Tools Used
- draw.io / diagrams.net (editable .drawio source for the concept map)
- Diagramming exported as SVG/PNG/PDF for the architecture and stack diagrams
- Word/PDF processor for this report

## Repository Structure
CSA15_CO1_AT1_<RegisterNumber>/
├─ README.md
├─ CSA15_CO1_AT1_<RegisterNumber>_Concept_Map.drawio
├─ evidence/
│   ├─ concept_map.png
│   ├─ service_model_matrix.pdf
│   ├─ decision_matrix.pdf
│   └─ architecture_diagram.png
└─ CSA15_CO1_AT1_<RegisterNumber>_Report.pdf

## CO1 Reflection
This assessment required applying cloud computing fundamentals — service models, deployment
models, virtualization, web services, and elasticity — to a concrete project rather than
describing them in the abstract. Mapping every Campus Connect module to a specific cloud service,
and justifying why, was what turned textbook definitions of IaaS/PaaS/SaaS/FaaS into an applied
design skill (CO1, BL2).
```